POLITECNICO DI TORINO

Master's Degree in Cybersecurity

Master's Degree Thesis

Penetration Test in Automotive Systems

Supervisor:
Prof. Alessandro Savino

Candidate:
Dennis D'Amico

Internship Tutor
TurinTech SE:
Eng. Mauro Bruno

Academic year 2025/2026

Abstract

LOREM ipsum
LOOK content/abstract.tex

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Prof. Alessandro Savino, for the time dedicated.
Special thanks goes to my company tutor, Ing. Mauro Bruno, and the entire team at TurinTech for their costant support.
Lastly, I am endlessly grateful to my family and friends for their company along the journey and for always cheering me up and believing in me even when I couldn't.

Table of Contents

1 Introduction to Automotive system and structure **1**

1.1 Evolution of Automotive Architecture 1

1.1.1 Structural Limitations and Network Evolution 1

1.2 Electronic Control Units 2

1.3 CAN Communication Protocol 2

2 Chapter Two: Unified Diagnostic Services Protocol **3**

2.1 Overview of ISO 14229 Standard 3

2.2 Client-Server Diagnostic Model 3

2.3 Key UDS Services and Request/Response Formats 3

2.4 Diagnostic Session Control 3

2.5 Security Access Mechanisms 3

3 Chapter Three: Threat Modeling and Vulnerability Analysis **5**

3.1 Automotive Threat Modeling 5

3.2 UDS protocol vulnerabilities 5

3.3 Attack Vectors on the Diagnostic Bus 5

4 Chapter Four: Experimental Laboratory Setup **7**

4.1 Hardware Testbed Architecture 7

4.2 Software Environment 7

5 Chapter Five: Implementation of Penetration Testing and Attacks **9**

5.1 Sniffing on CAN interface 9

5.2 Fuzzing on CAN protocol 9

5.3 Message injection 9

5.4 UDS Diagnostic Discovery 9

5.5 Seed Entropy 9

5.6 Replay Attack on Security Access 9

5.7 Parameter Discovery on Security Access 9

6 Chapter Six: Mitigation Strategies and Conclusions **11**

6.1 Secure Diagnostics via SecOC 11

6.2 Intrusion Detection and Prevention Systems 11

6.3 Final Remarks and Future Work 11

A Python Scripts **12**
A.1 UDS Diagnostic Discovery 12

Bibliography **15**

Dedications **16**

List of Figures

List of Tables

Acronyms

!!!	LOOK glossaries.tex.
ECU	Electronic Control Unit.
E/E	Electrical and Electronic.
ADAS	Advanced Driver Assistance Systems.
CAN	Controller Area Network.
CAN-FD	Controller Area Network - Flexible Datarate.

Chapter 1

Introduction to Automotive system and structure

1.1 Evolution of Automotive Architecture

For several decades the design of vehicles was mainly focused on providing a reliable mechanical infrastructure that would be both secure and economical. However with the advancement of embedded technology the paradigm shifted toward providing software-driven platforms that provide services to the user to enhance the commodity of the transportation.

To support this digital transition, the underlying Electrical and Electronic (E/E) architecture had to evolve through three main topological models [1]:

1. **Distributed architecture (The legacy model):** In early electronic systems, every new feature—such as anti-lock braking (ABS) or airbag deployment required a dedicated Electronic Control Unit (ECU). These units were in a closed network state and would not interact with external environments.

2. **Domain control architecture (The current standard):** In this division the architecture is organised into domains according to functionality. Typically, these domains include Powertrain, Chassis, Body, ADAS, and Infotainment.

3. **Zonal Architecture (The Next-Generation Paradigm):** The latest evolutionary step organises the vehicle's network based on physical location (e.g., front, rear, left, right) rather than logical function. Localised "Zonal Gateways" collect data from nearby components and stream it to a redundant central computer, drastically reducing weight and computational overhead.

1.1.1 Structural Limitations and Network Evolution

The continuous increase in the number of onboard ECUs quickly highlighted severe structural limitations within early E/E architectures, forcing the development of new in vehicle communication protocols:

- **From Traditional CAN to CAN-FD:** The classic Controller Area Network (CAN) bus has been the backbone of automotive networking for years. However, its limitation of maximum data rate of 1 Mbps and a payload capacity of only 8 bytes per frame became a critical bottleneck. With the increasing number of ECUs the bus suffered from heavy traffic saturation. This led to the introduction of CAN Flexible Data-rate (CAN-FD) [2], which mitigates these limits by boosting the data phase speed up to 5 Mbps and expanding the payload up to 64 bytes per frame, allowing more telemetry to be transmitted without delaying critical control messages.
- **The Transition to Automotive Ethernet:** Despite the bandwidth improvements of CAN-FD, modern data-heavy applications such as camera-based ADAS, advanced infotainment, and real-time sensors demand transmission speeds far beyond the capabilities of legacy buses. This has driven the adoption of Automotive Ethernet (standardised under IEEE 100BASE-T1 and 1000BASE-T1) [3], which provides high-bandwidth bidirectional communication (100 Mbps to 1 Gbps and beyond) over a single unshielded twisted pair of copper wires.

However, this transition from isolated, low-bandwidth networks to high-speed interconnected topologies has drastically modified the cyber security landscape. Protocols that were originally designed for closed, trusted environments are now exposed to internal and external entry points, turning diagnostic channels into critical surfaces for potential malicious exploits.

1.2 Electronic Control Units

1.3 CAN Communication Protocol

Chapter 2

Chapter Two: Unified Diagnostic Services Protocol

2.1 Overview of ISO 14229 Standard

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

2.2 Client-Server Diagnostic Model

2.3 Key UDS Services and Request/Response Formats

2.4 Diagnostic Session Control

2.5 Security Access Mechanisms

Chapter 3

Chapter Three: Threat Modeling and Vulnerability Analysis

3.1 Automotive Threat Modeling

3.2 UDS protocol vulnerabilities

3.3 Attack Vectors on the Diagnostic Bus

Chapter 4

Chapter Four: Experimental Laboratory Setup

4.1 Hardware Testbed Architecture

4.2 Software Environment

Chapter 5

Chapter Five: Implementation of Penetration Testing and Attacks

5.1 Sniffing on CAN interface

5.2 Fuzzing on CAN protocol

5.3 Message injection

5.4 UDS Diagnostic Discovery

5.5 Seed Entropy

5.6 Replay Attack on Security Access

5.7 Parameter Discovery on Security Access

Chapter 6

Chapter Six: Mitigation Strategies and Conclusions

6.1 Secure Diagnostics via SecOC

6.2 Intrusion Detection and Prevention Systems

6.3 Final Remarks and Future Work

Appendix A

Python Scripts

A.1 UDS Diagnostic Discovery

The complete script used to identify the Arbitration ID dedicated to Diagnostic is presented below:

Listing A.1: UDS Discovery

```
#CARINGCARIBOU DISCOVERY
import subprocess
from pathlib import Path
import can
import time

CURRENT_DIR = Path(__file__).resolve().parent
LOG_FILE_PATH = CURRENT_DIR / "logs/discovery_output.txt"
LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
#caringcaribou -i <interface> uds discovery -min <min_value> -max <max_value>

def verifica_coppia_diagnostica(client_id, server_id, interface="can0"):
    """
    send UDS DiagnosticSessionControl (10 01) to verify
    if the couple CLIENT/SERVER is diagnostic related or false positive.
    """
    is_extended = client_id > 0x7FF or server_id > 0x7FF
    can_mask = 0x1FFFFFFF if is_extended else 0x7FF
    can_filters = [{"can_id": server_id, "can_mask": can_mask, "extended": is_extended}]
    SERVICE = 0x10
    SUBFUNCTION = 0x01
    try:
        bus = can.interface.Bus(channel=interface, interface="socketcan", can_filters=can_filters)
        # Standard Payload Single Frame UDS (0x02 bytes, Service 0x10, Subfunction 0x01)
        payload = [0x02, SERVICE, SUBFUNCTION, 0xAA, 0xAA, 0xAA, 0xAA, 0xAA]
```

```
26          msg = can.Message(
27              arbitration_id=client_id,
28              data=payload,
29              is_extended_id=is_extended
30          )
31          bus.send(msg)
32          start_time = time.time()
33          while (time.time() - start_time) < 0.15:
34              rx_msg = bus.recv(timeout=0.05)
35              if rx_msg and rx_msg.arbitration_id == server_id:
36                  # If ECU responds with positive response (0x50) to
    service 0x10
37                  #single frame
38                  if len(rx_msg.data) >= 3 and rx_msg.data[1] == (SERVICE
     + 0x40) and rx_msg.data[2] == SUBFUNCTION:
39                      bus.shutdown()
40                      return True
41                  #first frame
42                  if len(rx_msg.data) >= 4 and ((rx_msg.data[0] & 0xF0)
    >> 4) == 0x01 and rx_msg.data[2] == (SERVICE + 0x40) and rx_msg.data
    [3] == SUBFUNCTION:
43                      bus.shutdown()
44                      return True
45          bus.shutdown()
46      except Exception as e:
47          print(f"\n[!] Errore durante il test hardware della coppia: {e}
    ")
48      return False
49
50  def run_discovery(min_value, max_value, interface="can0"):
51      """
52      Discovery using caringcaribou.
53      """
54      coppie_candidate = []
55      discovered_diagnostics = []
56
57      # Strings to saveCaring Caribou log
58      stderr_log = ""
59      print("[*] Caring Caribou loading...")
60      # Ask Linux to use caringcaribou
61      process = subprocess.Popen(
62          ['caringcaribou', '-i', interface, "uds", "discovery", "--min",
    str(min_value), "--max", str(max_value)],
63          stdout=subprocess.PIPE,
64          stderr=subprocess.PIPE,
65          text=True
66      )
67
68      # Capture standard output
69      for stdout_line in iter(process.stdout.readline, ""):
70          print(stdout_line, end="")  # Show original output
71          if "0x" in stdout_line and "|" in stdout_line:
72              parti = stdout_line.split("|")
```

```
73              if len(parti) >= 3:
74                  client_str = parti[1].strip()
75                  server_str = parti[2].strip()
76                  try:
77                      client_id = int(client_str, 16)
78                      server_id = int(server_str, 16)
79                      if (client_id, server_id) not in coppie_candidate:
80                          coppie_candidate.append((client_id, server_id))
81                  except ValueError:
82                      continue
83
84      # Check for false positives
85      print(f"\n[*] Found {len(coppie_candidate)} couples. Validation..."
        )
86
87      for client_id, server_id in coppie_candidate:
88          print(f"    [-] Test Client: 0x{client_id:08X} -> Server: 0x{
        server_id:08X} ... ", end="", flush=True)
89          if verifica_coppia_diagnostica(client_id, server_id, interface)
        :
90              print("VALID)")
91              discovered_diagnostics.append((client_id, server_id))
92          else:
93              print("False Positive")
94
95      # Print on screen
96      print(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} |")
97      print("-"*31)
98      for c_id, s_id in discovered_diagnostics:
99          print(f"| 0x{c_id:08x}   | 0x{s_id:08x}   |")
100     print("="*31)
101
102     # Print on file log
103     try:
104         with open(LOG_FILE_PATH, "w") as f:
105             f.write(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} |\n")
106             for c_id, s_id in discovered_diagnostics:
107                 f.write(f"| 0x{c_id:08x} | 0x{s_id:08x} |\n")
108     except Exception as e:
109         print(f"[ERR] Errore during save: {e}")
110     return discovered_diagnostics
111
112 if __name__ == "__main__":
113     #discovery brute force
114     min_value = 0x00000000
115     max_value = 0xFFFFFFFF
116     result = run_discovery(min_value=min_value, max_value=max_value,
        interface=interface)
117     with open(LOG_FILE_PATH_DIAGNOSTIC, "a") as f:
118         for client_id, server_id in result:
119             f.write(f"{hex(client_id)} {hex(server_id)}\n")
```

Bibliography

[1] EEWorld News. *Evolution of Automotive E/E Architecture: From Distributed to Zonal Centralised*. 2024. URL: `https://eeworld.com.cn` (visited on 07/02/2026) (cit. on p. 1).

[2] Robert Bosch GmbH. *CAN with Flexible Data-Rate Specification, Version 1.0*. White Paper. Robert Bosch GmbH, 2012 (cit. on p. 2).

[3] IEEE Computer Society. *IEEE Standard for Ethernet – Amendment 1: Physical Layer Specifications and Management Parameters for 100 Mb/s Operation over a Single Balanced Twisted Pair Cable (100BASE-T1)*. 2015 (cit. on p. 2).

Dedications

Dedications for everyone LOOK dedications.tex